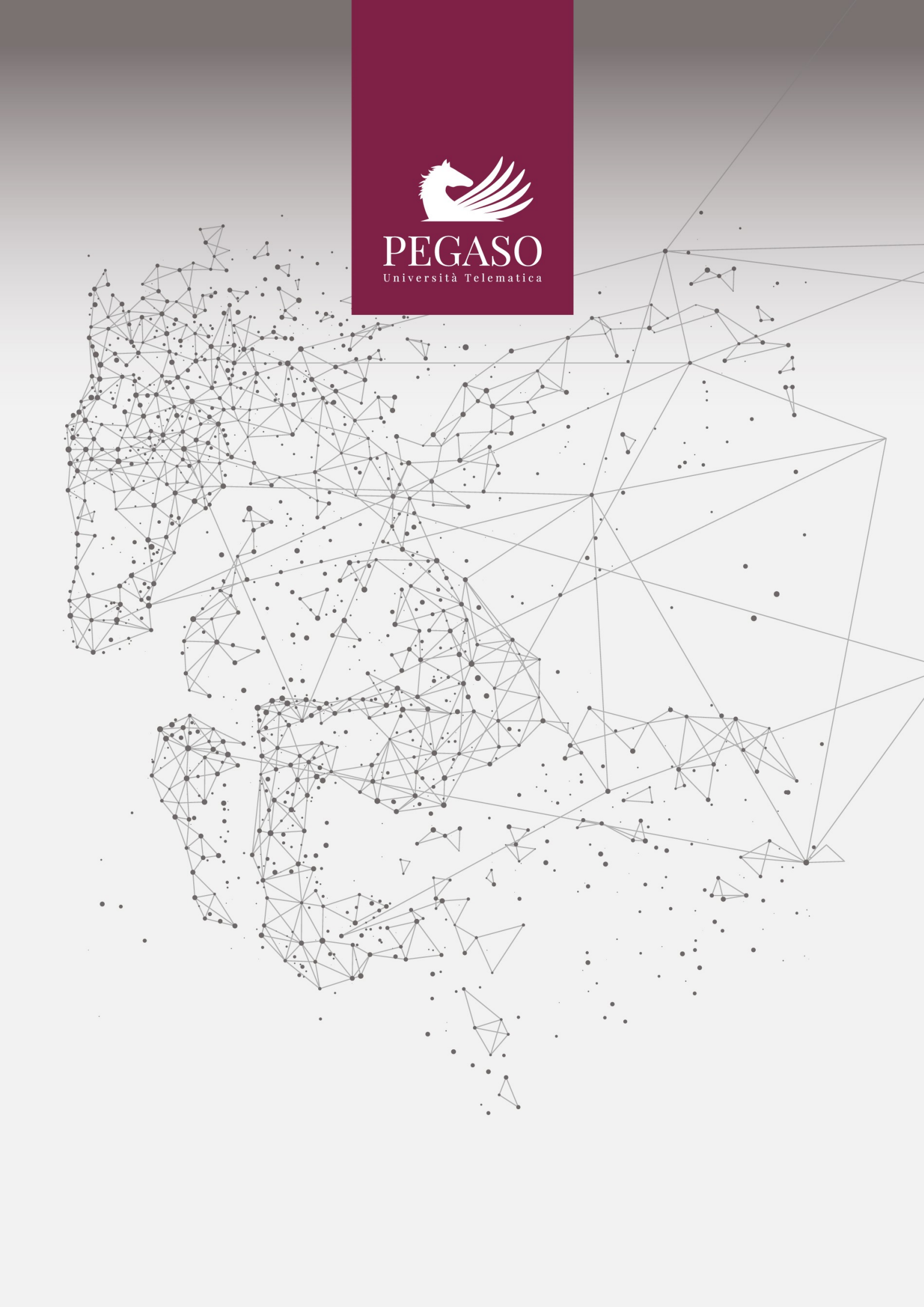

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. FORMATO DI TIPO I E INDIRIZZAMENTO TRAMITE BASE E OFFSET PER LW E SW 3

2. GESTIONE DEL TIPO DI DATO ARRAY 10

3. ESERCIZI 13

BIBLIOGRAFIA 16

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Formato di tipo i e indirizzamento tramite base e offset per lw e sw

La sequenza binaria che rappresenta una istruzione in LINGUAGGIO MACCHINA permette di ricavare tutte le informazioni che definiscono sia l'azione associata all'istruzione da eseguire, sia le modalità per il reperimento dei dati.

Queste informazioni sono codificate mediante due fondamentali nozioni che sono alla base della definizione di tutti i Linguaggi Macchina e che caratterizzano ogni istruzione:

- il Formato dell'istruzione,
- la Modalità di Indirizzamento dell'istruzione.

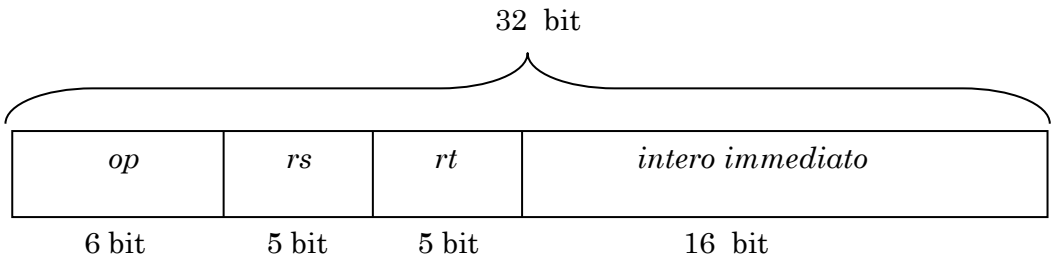
La SEQUENZA BINARIA che rappresenta l'**istruzione in Linguaggio Macchina** è suddivisa in sottosequenze, dette CAMPI, di lunghezza e posizione fissata i cui contenuti forniscono specifiche informazioni determinate univocamente dalle regole sintattiche dell'istruzione. Questa strutturazione, detta **FORMATO**, è solo una organizzazione concettuale e non determina nessuna suddivisione a livello fisico della sequenza.

Il **Formato di Tipo I** del Linguaggio Macchina MIPS deve la sua denominazione al fatto che un campo della sequenza binaria che rappresenta una istruzione con questo Formato contiene un numero intero rappresentato in Notazione in complemento a 2.

Questo numero fornisce il valore di un dato dell'istruzione detto **operando immediato**. Quindi la lettera I è l'iniziale della parola *Immediato*.

Il seguente schema indica la **partizione** concettuale (non fisica) in sottosequenze della sequenza di 32 bit dell'istruzione:

Formato di Tipo I



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

op: 6 bit, campo del *Codice Operativo* presente in tutte le istruzioni. Il valore contenuto indica l'operazione da eseguire ed il Formato dell'istruzione;

rs: 5 bit, indirizzo di registro;

rt: 5 bit, indirizzo di registro;

intero immediato: 16 bit, intero in Notazione in complemento a 2.

Gli indirizzi dei due registri sono dati da sequenze di 5 bit corrispondenti alla notazione simbolica in base alla seguente tabella:

simbolo	*indirizzo*	*simbolo*	*indirizzo*	*simbolo*	*indirizzo*
$t0	8	$s0	16	$t8	24
$t1	9	$s1	17	$t9	25
$t2	10	$s2	18		
$t3	11	$s3	19		
$t4	12	$s4	20		
$t5	13	$s5	21		
$t6	14	$s6	22		
$t7	15	$s7	23		

Le ISTRUZIONI DI TRASFERIMENTO DATI svolgono le operazioni di spostamento di valori tra la Memoria principale e i registri del processore gestendo accessi in **lettura** e **scrittura** tra questi blocchi di registri.

FORMATO delle istruzioni di TRASFERIMENTO DATI

Istruzione LOAD WORD in Linguaggio Macchina

L'istruzione LOAD WORD dell'Assembly MIPS LEGGE una parola dalla Memoria e ne scrive una copia in un registro.

Per evidenziare le informazioni che devono essere codificate nei campi dell'istruzione in Linguaggio Macchina, richiamiamo la forma sintattica dell'istruzione *load word* utilizzando un esempio:

```
lw  $t1,  7 ($s0)
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

lw Codice Operativo **lw** (**l**oad **w**ord) che specifica l'operazione,

$t1 l'**indirizzo del registro** in cui viene scritta la copia della parola LETTA dalla Memoria,

7 **OFFSET** (spiazzamento), utilizzato per il calcolo dell'indirizzo di accesso in lettura alla Memoria,

$s0 **indirizzo del registro** detto **REGISTRO BASE** che contiene un **indirizzo di Memoria** utilizzato per il calcolo dell'indirizzo di accesso in lettura alla Memoria.

L'istruzione in LINGUAGGIO MACCHINA MIPS che traduce l'istruzione Assembly MIPS *lw* è caratterizzata da:

- Formato di **Tipo I**,
- Modalità di **Indirizzamento tramite Base e Offset**.

L'operando immediato dell'istruzione è il valore dell'Offset, che nell'esempio è dato dal numero 7, e nel Formato di Tipo I occupa il campo di 16 bit.

Il Formato di Tipo I mette a disposizione 2 campi di lunghezza 5 utilizzabili per gli indirizzi dei registri, che nell'esempio sono:

$t1 ⟷ 9 ⟷ 01001

$s0 ⟷ 16 ⟷ 10000.

Il Codice Operativo è codificato nel campo di 6 bit comune a tutti i Formati. Per l'istruzione *lw* il campo *Codice Operativo* contiene il **valore decimale 35**, rappresentato in Notazione binaria dalla sequenza 100011, e fornisce tutte le informazioni che specificano l'operazione da eseguire ed il Formato dell'istruzione.

Tutte queste informazioni occupano specifici campi del Formato di Tipo I componendo la sequenza di 32 bit che rappresenta l'istruzione in Linguaggio Macchina. L'ordine degli indirizzi dei registri viene invertito come mostrato dal seguente schema:

lw $t1, 7 ($s0)

op	*rs*	*rt*	*intero immediato*
35	16	9	7

la cui rappresentazione binaria è:

100011	10000	01001	0000000000000111

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

e quindi la sequenza dei 32 bit in Linguaggio Macchina è:

10001110000010010000000000000111

Istruzione STORE WORD in Linguaggio Macchina

L'operazione di trasferimento dati inversa rispetto a *load word* è eseguita dalla istruzione STORE WORD dell'ASSEMBLY MIPS, che SCRIVE in Memoria la parola contenuta in un registro, ed ha una forma sintattica analoga.

Per evidenziare le informazioni che devono essere codificate nei campi dell'istruzione in Linguaggio Macchina, richiamiamo la forma sintattica dell'istruzione *store word* utilizzando un esempio:

sw $s5, 1 ($t2)

sw Codice Operativo **sw** (**s**tore **w**ord) che specifica l'operazione,

$s5 l'**indirizzo del registro** da cui viene letta la parola da scrivere in Memoria,

1 **OFFSET** (spiazzamento), utilizzato per il calcolo dell'indirizzo di accesso in scrittura alla Memoria,

$t2 **indirizzo del registro** detto **REGISTRO BASE** che contiene un **indirizzo di Memoria** utilizzato per il calcolo dell'indirizzo di accesso in scrittura alla Memoria.

Analogamente a *load word*, anche l'istruzione STORE WORD che traduce l'istruzione Assembly *sw* in LINGUAGGIO MACCHINA MIPS è caratterizzata da:

- Formato di **Tipo I**,
- Modalità di **Indirizzamento tramite Base e Offset**.

Il Codice Operativo dell'istruzione in Linguaggio Macchina è codificato nel campo di 6 bit comune a tutti i Formati. Per l'istruzione *sw* il campo *Codice Operativo* contiene il **valore decimale 43**, rappresentato in Notazione binaria dalla sequenza 101011, e fornisce tutte le informazioni che specificano l'operazione da eseguire ed il Formato dell'istruzione.

La traduzione del precedente esempio si ottiene applicando le stesse regole sintattiche descritte per *lw*, ricordando che gli indirizzi dei registri sono dati dai valori:

$s5 ⟷ 21 ⟷ 10101

$t2 ⟷ 10 ⟷ 01010

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

sw $s5, 1 ($t2)

op	*rs*	*rt*	*intero immediato*
43	10	21	1

la rappresentazione binaria è:

101011	01010	10101	0000000000000001

e quindi la sequenza dei 32 bit in Linguaggio Macchina è:

10101101010101010000000000000001

MODALITÀ DI INDIRIZZAMENTO TRAMITE BASE E OFFSET

L'istruzione contiene in maniera completa e non ambigua tutte le informazioni necessarie alla sua esecuzione. Un aspetto importante riguarda il come reperire i dati coinvolti nell'esecuzione.

Questa informazione è ottenuta mediante la **Modalità di Indirizzamento** associata all'istruzione che stabilisce le regole per determinare gli indirizzi dei dati coinvolti nell'esecuzione, mediante il contenuto dei campi stabiliti dal Formato dell'istruzione.

Le istruzioni *lw* e *sw* effettuano un accesso ai registri della Memoria principale per LEGGERE (*lw*) oppure per SCRIVERE (*sw*) una *parola* costituita da 4 Byte consecutivi.

La Modalità di Indirizzamento è la stessa per entrambe le istruzioni *lw* e *sw*.

L'***Indirizzamento tramite Base e Offset*** stabilisce la **regola** per il calcolo dell'indirizzo del primo Byte della parola che, in base all'approccio *Big-endian*, rappresenta i valori degli 8 bit più significativi della parola da leggere o scrivere in Memoria:

INDIRIZZAMENTO TRAMITE BASE E OFFSET

Indirizzo di Memoria =

contenuto del *Registro Base* + *Offset* **esteso di segno** a 32 bit

L'ESTENSIONE DEL SEGNO del valore dell'*Offset* contenuto nel *campo immediato* di 16 bit è necessaria in quanto l'addizione è eseguita dalla Unità Aritmetico-Logica (ALU) che richiede operandi di 32 bit. L'estensione del segno aggiunge i bit a sinistra della sequenza ponendo il loro valore uguale a quello del bit di segno. In base alle proprietà della Notazione in complemento a 2, l'estensione del segno non cambia il valore rappresentato.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

OSSERVAZIONE: Quando l'Offset ha valore 0 l'indirizzo di accesso alla Memoria coincide con l'indirizzo Base contenuto nel registro Base.

Illustriamo questa Metodologia di Indirizzamento, analoga per *load word* e *store word*, considerando un esempio relativo a *lw*:

```
lw  $t1,  7 ($s0)
```

Il Registro Base ha l'indirizzo $s0, cioè 16, riportato nel campo *rs*. Il contenuto di tale registro è la sequenza di 32 bit che rappresenta l'indirizzo di Memoria (Base di partenza) a cui va addizionato l'Offset (spiazzamento) dato dal numero 7, riportato in Notazione in complemento a 2 nei 16 bit del campo *intero immediato*, che prima dell'addizione viene esteso di segno a 32 bit.

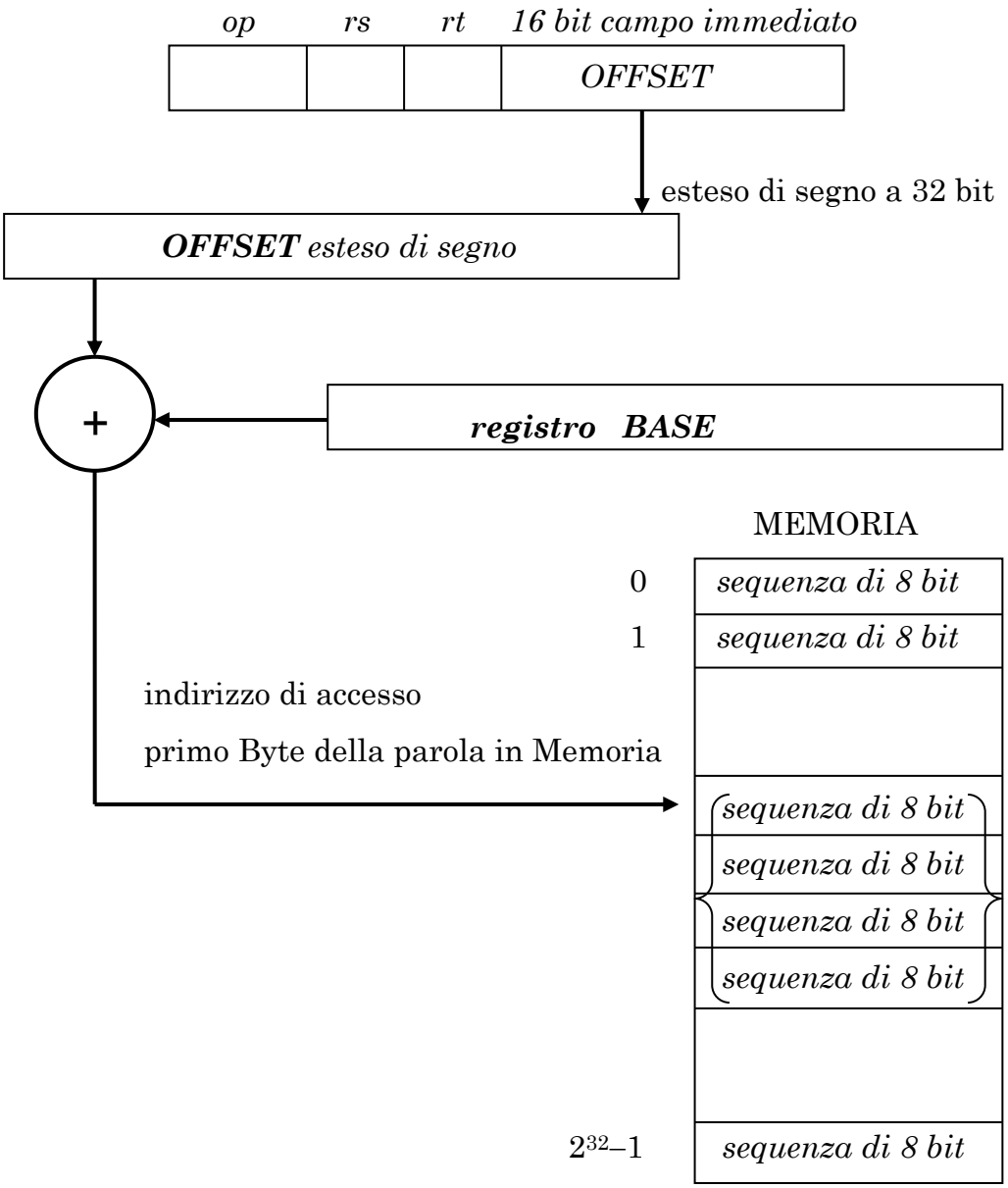
La somma fornisce l'indirizzo di accesso in Memoria da cui iniziano i 4 Byte della parola da leggere per scriverla nel registro del processore il cui indirizzo è $t1, cioè 9, riportato nel campo *rt.*

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La seguente figura fornisce una rappresentazione schematica della Modalità di Indirizzamento tramite Base e Offset:

MODALITÀ di Indirizzamento tramite Base e Offset



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Gestione del Tipo di dato Array

Oltre ai Tipi di dati primitivi come i numeri interi, i Linguaggi ad Alto Livello forniscono supporto per la gestione di alcuni Tipi di dati strutturati. Uno dei più importanti è il *Tipo di dato Array*.

La struttura dati Array è una lista di elementi in cui ogni elemento è direttamente accessibile tramite un *indice* ad esso associato. L'indice è un numero intero che tipicamente inizia dal valore 0. La notazione generalmente utilizzata per indicare uno specifico elemento dell'Array e costituita dal *nome* dell'Array seguito dall'indice che individua l'elemento racchiuso tra parentesi.

Gli elementi di un Array sono tutti dello stesso Tipo e la *dimensione* dell'Array stabilisce il numero degli elementi che lo costituiscono. Queste informazioni sono esplicitate nella dichiarazione di Tipo.

Il seguente esempio è una dichiarazione in Linguaggio C di un Array di nome A costituito da 10 elementi dati da numeri interi:

```
int  A[10] ;
```

poiché il valore dell'indice inizia da 0, i valori degli indici di questo Array variano da 0 a 9, quindi l'ultimo elemento dell'Array è A[9]:

A[0], A[1], A[2], A[3], A[4], A[5], A[6], A[7], A[8], A[9].

Un errore comune e dimenticare questa convenzione e utilizzare erroneamente l'indice 10 per indicare l'ultimo elemento dell'Array.

Questo errore è detto ***Array out of bounds*** (fuori dai limiti), perché porta ad accedere a locazioni di Memoria fuori dallo spazio destinato a contenere gli elementi dell'Array che tipicamente sono memorizzati in locazioni successive di Memoria.

La strutturazione del Linguaggio di programmazione ad alto Livello può prevedere un controllo automatico sui limiti degli indici per evitare questo errore. Ad esempio, il Linguaggio Java prevede questo controllo, invece i Linguaggi C e C++ non lo prevedono.

Lo spazio di Memoria occupato da un Array dipende sia dalla dimensione dell'Array, cioè dal numero di elementi, sia dal Tipo degli elementi.

Ad esempio, se gli elementi sono numeri interi, ogni elemento è rappresentato da una parola, cioè una sequenza di 32 bit, quindi occorrono 4 Byte, ovvero 4 locazioni di Memoria per memorizzare ciascun elemento dell'Array.

Quindi con la dichiarazione di Tipo del precedente esempio:

```
int  A[10] ;
```

è necessario uno spazio di Memoria pari a 4x10 = 40 Byte per memorizzare gli elementi dell'Array.

PEGASO
Università Telematica

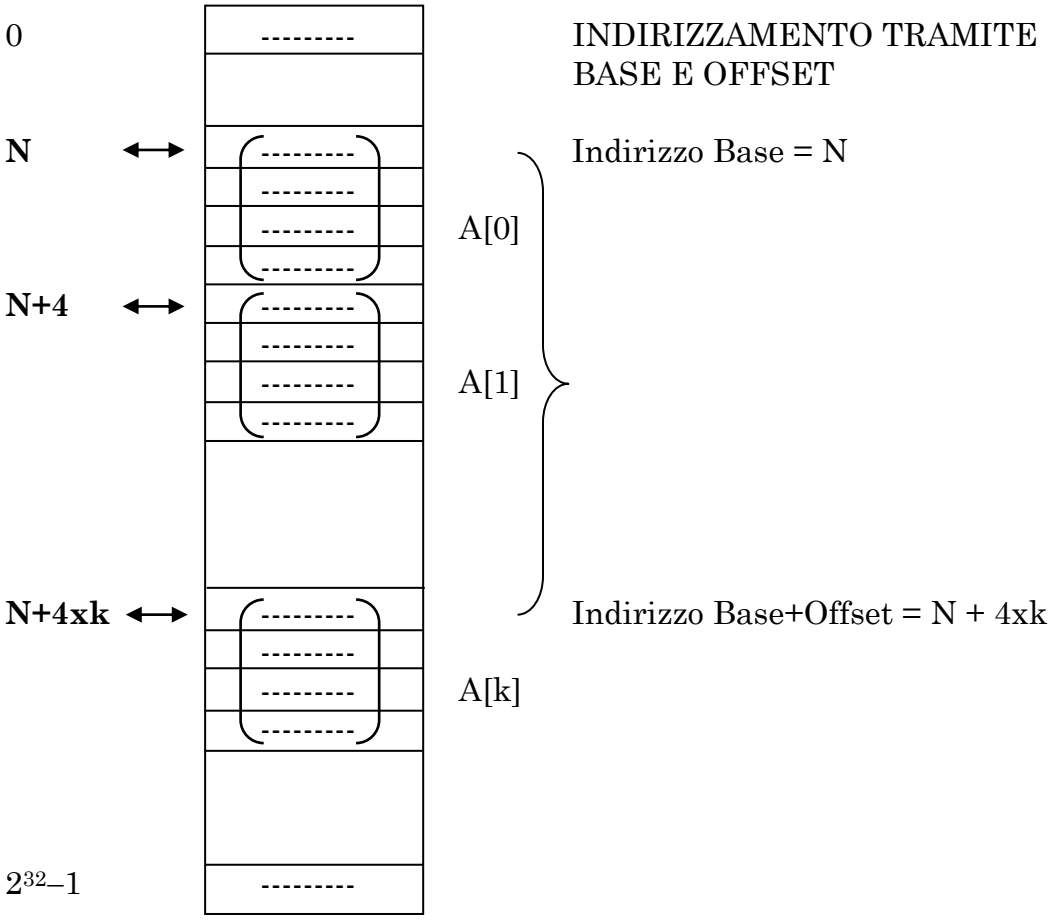
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La gestione di un Array prevede accessi in Memoria in LETTURA e SCRITTURA connessi con l'elaborazione dei dati in esso contenuti, quindi nella traduzione in Assembly sono coinvolte le istruzioni di trasferimento dati *load* e *store* che utilizzano la Metodologia di Indirizzamento tramite Base e Offset per il calcolo dell'indirizzo di accesso in Memoria.

Questo porta in modo naturale ad assumere l'indirizzo del primo Byte del primo elemento di un Array come INDIRIZZO BASE per gli accessi in Memoria utilizzando l'OFFSET per la gestione degli indici dell'Array.

Infatti, per accedere all'elemento A[k] dell'Array, individuato tramite il relativo indice k, è necessario determinare l'indirizzo di Memoria del primo Byte dei quattro che costituiscono la parola di Memoria in cui l'elemento è memorizzato. Questo indirizzo può essere calcolato aggiungendo all'indirizzo relativo al primo Byte del primo elemento A[0] dell'Array, un Offset pari a 4xk, vale a dire il multiplo di 4 corrispondente al valore k dell'indice dell'elemento cui accedere.

Il seguente schema illustra questa modalità di gestione degli indici per un Array costituito da elementi rappresentati da sequenze di 32 bit, cioè da 4 Byte. Nell'esempio N rappresenta il valore dell'Indirizzo Base dell'Array:



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

L'indirizzo della locazione di Memoria da cui parte lo spazio occupato dagli elementi di un Array dato dal **primo Byte dell'elemento A[0]** è detto **Indirizzo Base dell'Array**, e costituisce l'informazione fondamentale per la gestione di questo Tipo di dato.

Questo indirizzo viene utilizzato anche per passare i dati dell'Array nelle chiamate di procedura nel Linguaggio C, che mette a disposizione il Tipo di dato puntatore per la gestione degli indirizzi.

Il linguaggio C++, pur essendo un linguaggio strutturato per il paradigma *object oriented*, include i Tipi di dato Array e puntatore ereditati dal Linguaggio C, che riflettono la separazione fra i dati e gli algoritmi che operano sui dati, caratteristica del paradigma *procedurale*.

In Java, Linguaggio completamente basato sul paradigma *object oriented*, il Tipo di dato puntatore non è disponibile e gli Array sono oggetti i cui elementi sono gestiti *per riferimento* tramite l'indice.

Tuttavia, dal punto di vista implementativo, l'indice è comunque strettamente connesso alla specificazione dell'Offset rispetto all'indirizzo Base dell'Array assegnato in fase di compilazione.

Le parentesi quadre usate per indicare l'indice sono trattate in Java come un operatore che ha la priorità più alta tra tutti gli operatori del linguaggio. Questo operatore effettua anche il controllo automatico per evitare l'errore di *Array out of bounds*. Se l'indice utilizzato per accedere all'elemento dell'Array non è un riferimento valido viene generata l'*eccezione* detta ArrayIndexOutOfBound.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Esercizi

ESERCIZIO

Data la dichiarazione di Array e l'istruzione di assegnamento in Linguaggio C:

```
int  A[30], B[30];
B[12] =  A[5] + 7;
```

- **indicare il numero di locazioni assegnato ad ogni Array,**
- **tradurre in Assembly MIPS,**
- **tradurre in Linguaggio Macchina MIPS.**

Entrambi gli Array contengono numeri interi memorizzati mediante sequenze di 32 bit, quindi ogni elemento occupa 4 Byte di Memoria. Gli Array hanno la stessa dimensione di 30 elementi, ne segue che ciascun Array occupa:

30 x 4 = 120 Byte.

Per determinare la traduzione in Linguaggio Assembly MIPS fissiamo i seguenti indirizzi per i registri che contengono gli Indirizzi Base dei due Array:

A[ ]	B[ ]	
$t1	$s0	*notazione simbolica*
9	16	*notazione decimale*
01001	10000	*notazione binaria*

Per il calcolo della somma che figura nell'istruzione è necessario leggere l'operando contenuto in Memoria nell'elemento A[5] con una istruzione *load word* copiandone il valore in un registro.

L'Offset che consente di ottenere l'indirizzo del primo Byte della parola di Memoria dove è memorizzato l'elemento A[5] è dato dal valore 4x5=20:

```
lw  $t3, 20 ($t1)        # copia nel registro $t3 il valore A[5]
```

quindi si può effettuare la somma con l'operando immediato 7:

```
addi  $t6, $t3, 7        # il registro $t6 contiene il valore A[5] + 7.
```

L'assegnamento indicato nell'istruzione in Linguaggio C viene effettuato scrivendo in Memoria il risultato ottenuto dalla somma nell'elemento B[12], utilizzando l'istruzione *store word*.

L'Offset che consente di ottenere l'indirizzo del primo Byte della parola di Memoria dove è memorizzato l'elemento B[12] è dato dal valore 4x12=48:

```
sw  $t6, 48 ($s0)        # B[12] contiene il valore A[5] + 7
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Quindi la traduzione in Linguaggio Assembly MIPS è:

lw $t3, 20 ($t1)

addi $t6, $t3, 7

sw $t6, 48 ($s0)

Per determinare la traduzione in Linguaggio Macchina MIPS ricordiamo che i Codici Operativi delle istruzioni corrispondenti sono:

addi	lw	sw	
8	35	43	*notazione decimale*
001000	100011	101011	*notazione binaria*

e che gli indirizzi dei registri di supporto utilizzati sono:

$t3	$t6	*notazione simbolica*
11	14	*notazione decimale*
01011	01110	*notazione binaria*

Le istruzioni in Linguaggio Macchina utilizzano tutte il Formato di Tipo I, e in Notazione decimale sono:

35	9	11	20
8	11	14	7
43	16	14	48

la cui rappresentazione binaria è:

100011	01001	01011	0000000000010100
001000	01011	01110	0000000000000111
101011	10000	01110	0000000000110000

Quindi, la traduzione in Linguaggio Macchina MIPS è:

10001101001010110000000000010100

00100001011011100000000000000111

10101110000011100000000000110000

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La seguente Tabella riassume in forma schematica le regole sintattiche delle istruzioni Aritmetico-Logiche e di Trasferimento dati in Linguaggio Macchina MIPS:

Linguaggio Macchina MIPS

		6 bit	5 bit	5 bit	5 bit	5 bit	6 bit	esempio
istruzione	*Tipo*	*op*	*rs*	*rt*	*rd*	*shamt*	*funct*	
add	R	0	$s1	$s2	$t0	0	32	add $t0, $s1, $s2
sub	R	0	$s1	$s2	$t0	0	34	sub $t0, $s1, $s2
and	R	0	$s1	$s2	$t0	0	36	and $t0, $s1, $s2
or	R	0	$s1	$s2	$t0	0	37	or $t0, $s1, $s2
sll	R	0	0	$s2	$t0	7	0	sll $t0, $s2, 7
srl	R	0	0	$s2	$t0	7	2	srl $t0, $s2, 7
addi	I	8	$s3	$t0	–9			addi $t0, $s3,–9
lw	I	35	$s3	$t0	–9			lw $t0, –9($s3)
sw	I	43	$s3	$t0	–9			sw $t0, –9($s3)

le relative Modalità di indirizzamento sono:

istruzione	*Formato Tipo*	*Modalità di Indirizzamento*
add	R	tramite registro
sub	R	tramite registro
and	R	tramite registro
or	R	tramite registro
sll	R	tramite registro
srl	R	tramite registro
addi	I	immediato
lw	I	tramite Base e Offset
sw	I	tramite Base e Offset

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- D. A. Patterson, J. L. Hennessy, "Struttura e progetto dei calcolatori", Zanichelli.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).